CMPE 321 — Project 3: Modular DBMS Engine

Group 28 | Ökkeš Berkay Acer (2022400066) | Enes Öztürk (2022400042) | Boğaziçi University, Spring 2026

Implementation

We built a four-layer Python DBMS engine: **Disk Space Manager** (binary fixed-size page I/O, read/write counters), **Buffer Manager** (in-memory page cache with LRU/MRU eviction, hit/miss tracking), **File & Index Manager** (slotted pages, persistent catalog, three interchangeable index strategies), and **Query Processor** (command parser, `output.txt` writer, `log.csv`, `explain`/`stats` commands). The single entry point `archive.py` drives the engine; all parameters live in `config.json`. The project ships with 44 passing unit tests and a workload generator used in experiments.

Design Choices & Justification

Typed Result objects across layers. No raw bytes or integers cross a layer boundary — every call returns a structured Result. This made the 44-test suite tractable: each layer can be unit-tested in isolation without mocking the layers below it.

B$^+$-tree as the default index. We chose B$^+$-tree over pure hash because it supports both point lookups and range queries with only marginal I/O overhead (Exp. 2: 10 375 reads vs. 10 177 for hash). The hash index uses FNV-1a with overflow chaining; it is faster on pure point workloads but cannot serve range scans.

LRU as the default replacement policy. Experiment 1 shows LRU dominates MRU on both sequential (97.4% vs. 5.0% hit rate) and random workloads (76.1% vs. 13.8%). MRU evicts the most-recently-used page immediately, which performs well only on one-time full scans — a rare case here.

Slotted pages. Slotted-page layout allows variable-length records and in-page compaction without moving record IDs, making the record layer independent of physical page layout.

Experiment Results

Exp. 1 — LRU vs. MRU (pool = 16, B$^+$-tree, 1 000 records / 200 queries):

Workload + Policy	Disk Reads	Disk Writes	Hit Rate	Evictions
Sequential + LRU	21	2	97.4%	21
Sequential + MRU	570	0	5.0%	570
Random + LRU	191	2	76.1%	191
Random + MRU	414	0	13.8%	414

Exp. 2 — Index Strategies (pool = 16, LRU, mixed workload):

Strategy	Disk Reads	Disk Writes	Nodes	Recs Scanned	Returned
heap_scan	14 901	1	—	149 375	10 469
hash_index	10 177	2	100	100 000	10 469
bplus_tree	10 375	2	300	100 000	10 469

Heap scan reads 46% more pages than the indexed strategies due to full-table traversal. Hash and B$^+$-tree achieve identical record throughput with comparable I/O; B$^+$-tree visits more nodes (300 vs. 100) but remains competitive because tree nodes fit in buffer pages.

Exp. 3 — Buffer Pool Size (LRU, B$^+$-tree, random workload):

Pool Size	Disk Reads	Hit Rate	Evictions
4	349	56.4%	349
8	282	64.8%	282
16	191	76.1%	191
32	148	81.5%	148
64	85	89.4%	85

Hit rate improves monotonically but with diminishing returns above pool = 32; doubling from 32 to 64 saves only 63 reads while doubling from 4 to 8 saves 67.

GitHub: https://github.com/Berkayacer13/project3

Explanation Video: https://drive.google.com/file/video